

BAŞLIK: HTTP PROTOKOLÜ: WEB API'LERİNİN TAŞIMA KATMANI

TÜR: Akademik/Teknik Detaylı Anlatım

BÖLÜM 1: HTTP NEDİR VE NEDEN VAR?

HTTP (HyperText Transfer Protocol), istemci ve sunucu arasında veri alışverişini standartlaştıran, 1991'de Tim Berners-Lee tarafından tasarlanan bir iletişim protokolüdür. Bugün internetteki hemen her Web API, temelinde HTTP kullanır — bu yüzden HTTP'yi anlamadan REST API tasarlamak veya tüketmek mümkün değildir. HTTP, OSI ağ modelinde "Uygulama Katmanı"nda (Application Layer) çalışır ve altında TCP/IP protokolüne (veri paketlerinin güvenilir şekilde iletilmesini sağlayan katman) dayanır.

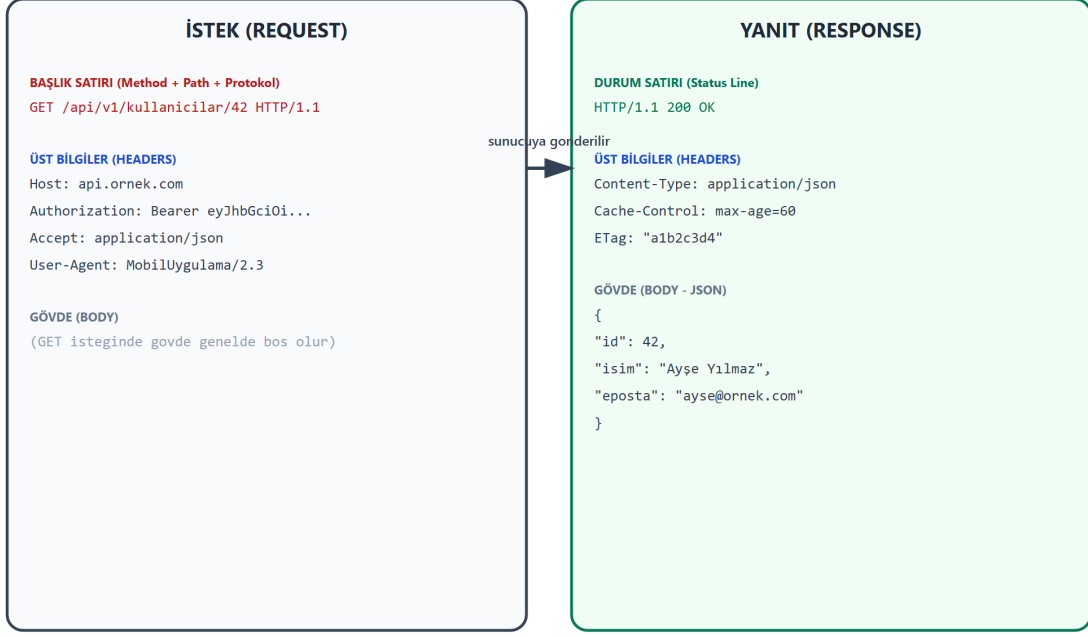
1.1. HTTP'nin Durumsuzluğu (Statelessness)

HTTP'nin en temel — ve en sık yanlış anlaşılan — özelliği DURUMSUZ (stateless) olmasıdır: sunucu, bir istekten diğerine HİÇBİR BİLGİYİ HATIRLAMAZ. Her istek, sunucu için sanki ilk kez geliyormuş gibi bağımsız olarak işlenir; önceki istekler hakkında hiçbir "hafıza" yoktur. Bu kulağa garip gelebilir — peki bir web sitesine giriş yaptığınızda sunucu sizi nasıl "hatırlıyor"? Cevap: her istekle birlikte, kimliğinizi kanıtlayan bir bilgi (çerez/cookie, ya da bir Authorization başlığındaki token) YENİDEN gönderilir; sunucu her seferinde bu bilgiyi kontrol eder, ama bunu kendi hafızasında tutmaz. Durumsuzluk, API'lerin yatay olarak ölçeklenebilmesinin (horizontal scaling) da temelidir: herhangi bir istek, arka planda yüzlerce sunucudan HERHANGİ birine yönlendirilebilir, çünkü hiçbir sunucunun "hatırlaması gereken" bir şey yoktur.

BÖLÜM 2: BİR HTTP İSTEK/YANIT ÇİFTİNİN ANATOMİSİ

Her HTTP iletişimi bir İSTEK (request) ve buna karşılık gelen bir YANIT (response) çiftinden oluşur. İkisi de aynı üç parçalı yapıyı takip eder: bir başlık/durum satırı, üst bilgiler (headers) ve isteğe bağlı bir gövde (body).

Bir HTTP İstek ve Yanıtının Anatomisi



Şekil 1 — Solda tipik bir istek, sağda buna karşılık gelen yanıt. Her ikisi de başlık, üst bilgiler ve (isteğe bağlı) gövdeden oluşur.

2.1. İstek (Request) Bileşenleri

- Method (Fiil): İsteğin NİYETİNİ belirtir (GET, POST, PUT, PATCH, DELETE — bkz. Bölüm 3).
- Path (Yol): Hangi kaynağa (resource) erişilmek istendiğini belirtir (/api/v1/kullanıcılar/42).
- Query String (Sorgu Parametreleri): URL'nin sonuna ? ile eklenen, filtreleme/sayfalama gibi ek bilgiler taşıyan anahtar-değer çiftleri (?sayfa=2&limit;=20).
- Headers (Üst Bilgiler): İstek hakkında meta veri taşır — kimlik doğrulama bilgisi (Authorization), istemcinin kabul ettiği format (Accept), istemci kimliği (User-Agent).
- Body (Gövde): İstekle birlikte gönderilen asıl veri (genellikle POST/PUT/PATCH isteklerinde bulunur; GET isteklerinde teknik olarak mümkün olsa da pratikte neredeyse hiç kullanılmaz).

2.2. Yanıt (Response) Bileşenleri

- Status Line (Durum Satırı): İsteğin sonucunu 3 haneli bir kodla özetler (200 OK, 404 Not Found — bkz. Bölüm 4).
- Headers: Yanıt hakkında meta veri (içerik tipi, önbellekleme kuralları, sürüm etiketi ETag).
- Body: İstenen asıl veri (genellikle JSON formatında).

BÖLÜM 3: HTTP METODLARI (FİİLLER) VE SEMANTİKLERİ

HTTP metodları, bir isteğin sunucudan NE YAPMASINI istediğini belirtir. İyi tasarlanmış bir API, bu fiilleri doğru semantikleriyle kullanır (bkz. 22.3-REST_Mimarisi_ve_Tasarımı, Richardson Olgunluk Modeli Seviye 2).

- **GET**: Bir kaynağı OKUR, hiçbir şeyi değiştirmez. Örnek: GET /urunler/5 — 5 numaralı ürünün bilgisini getirir.
- **POST**: Yeni bir kaynak OLUŞTURUR (veya genel bir işlem tetikler). Örnek: POST /urunler — yeni bir ürün ekler.
- **PUT**: Bir kaynağın TAMAMINI değiştirir (gönderilmeyen alanlar sıfırlanır/silinir). Örnek: PUT /urunler/5 — 5 numaralı ürünün TÜM bilgilerini yeni gönderilen verilerle değiştirir.
- **PATCH**: Bir kaynağın SADECE belirtilen alanlarını günceller. Örnek: PATCH /urunler/5 ile gövdede sadece {"fiyat": 199} gönderilirse, SADECE fiyat değişir, diğer alanlar aynı kalır.
- **DELETE**: Bir kaynağı SİLER. Örnek: DELETE /urunler/5.
- **HEAD**: GET ile aynıdır ama sadece başlıkları döner, gövdeyi döndürmez (bir kaynağın var olup olmadığını, boyutunu hızlıca kontrol etmek için kullanılır).
- **OPTIONS**: Bir kaynağın hangi metodları desteklediğini sorar (CORS ön-uçuş / preflight isteklerinde tarayıcılar tarafından otomatik kullanılır).

3.1. Güvenlik (Safety) ve İdempotentlik (Idempotency) Kavramları

İki kritik teorik kavram, API tasarımında hangi metodun ne zaman kullanılacağını belirler:

- **Güvenli (Safe) Metod**: Sunucudaki veriyi HİÇ DEĞİŞTİRMEYEN metodlardır. Sadece GET, HEAD ve OPTIONS güvenlidir.
- **İdempotent Metod**: Aynı isteği 1 kere de gönderseniz, 100 kere de gönderseniz, sunucudaki NİHAİ durum aynı olur. GET, PUT, DELETE idempotenttir (bir kaydı 5 kere silmek de, 1 kere silmek de aynı sonucu — kaydın silinmiş olması — verir). **POST İDEMPOTENT DEĞİLDİR**: aynı "yeni sipariş oluştur" isteğini 2 kere gönderirseniz, 2 ayrı sipariş oluşur! Bu, ödeme sistemleri gibi kritik API'lerde ağ hatası nedeniyle bir isteğin YANLIŞLIKLA iki kez gönderilmesi durumunda ciddi hatalara (çift ödeme alma) yol açabilir — bu yüzden production ödeme API'leri genellikle bir "Idempotency-Key" başlığı kullanarak POST isteklerini de idempotent hale getirir.

PUT idempotenttir (aynı veriyle 5 kere PUT yaparsanız, sonuçta kaynak hep aynı son değerdedir), PATCH ise genellikle idempotent DEĞİLDİR (örneğin {"stok": stok - 1} gibi göreceli bir güncelleme yapan bir PATCH isteği, tekrar gönderildiğinde stoğu bir kez daha azaltır).

BÖLÜM 4: HTTP DURUM KODLARI (STATUS CODES)

Durum kodları, 3 haneli sayılardır ve ilk hanesine göre 5 aileye ayrılır. Bir API tüketen geliştirici olarak (bkz. 22.5) veya bir API tasarlayan geliştirici olarak (bkz. 22.6), doğru durum kodunu doğru anlamda kullanmak/beklemek hayati önemdedir.

HTTP Durum Kodu Aileleri (Status Code Categories)

1xx	BİLGİLENDİRME Informational	100 Continue
2xx	BASARILI Success	200 OK • 201 Created • 204 No Content
3xx	YONLENDİRME Redirection	301 Moved • 304 Not Modified
4xx	İSTEMCI HATASI Client Error	400 Bad Request • 401 • 403 • 404 • 429
5xx	SUNUCU HATASI Server Error	500 Internal • 502 Bad Gateway • 503

Şekil 2 — HTTP durum kodu aileleri ve en sık karşılaşılan örnekleri.

En sık karşılaşılan ve kritik kodlar: **200 OK** (başarılı), **201 Created** (yeni kaynak başarıyla oluşturuldu — genellikle POST'un başarılı yanıtı), **204 No Content** (işlem başarılı ama dönecek veri yok — genellikle DELETE'in başarılı yanıtı), **400 Bad Request** (istemci hatalı/eksik veri gönderdi), **401 Unauthorized** (kimlik doğrulama YOK ya da geçersiz — "sen kimsin bilmiyorum"), **403 Forbidden** (kimliğin biliniyor ama bu işlem için yetkin yok — "seni tanıyorum ama izin veremem", bkz. 22.4), **404 Not Found** (kaynak yok), **429 Too Many Requests** (hız sınırı aşıldı, bkz. 22.4), **500 Internal Server Error** (sunucu tarafında beklenmedik bir hata oluştu — istemcinin hatası DEĞİLDİR).

Sık yapılan bir tasarım hatası: her hatada 200 OK dönüp, hatanın detayını sadece gövde içine yazmaktır ("200 OK, body: {'hata': 'kullanıcı bulunamadı'}"). Bu, HTTP'nin sunduğu standart hata iletişim kanalını (status code) kullanmamak anlamına gelir ve istemci tarafında (özellikle otomatik hata yönetimi/retry mantığında, bkz. 22.5) gereksiz karmaşaya yol açar. İyi pratik: HER ZAMAN anlamlı durum kodunu kullanın, gövdeyi sadece EK detay için kullanın.

BÖLÜM 5: ÖNEMLİ HTTP ÜST BİLGİLERİ (HEADERS)

- **Content-Type**: Gövdenin formatını belirtir (application/json, multipart/form-data — dosya yüklemede, text/html).
- **Authorization**: Kimlik doğrulama bilgisini taşır (Bearer <token>, bkz. 22.4).
- **Accept**: İstemcinin hangi formatta yanıt kabul ettiğini belirtir.
- **Cache-Control**: Yanıtın ne kadar süreyle önbelleklenebileceğini belirtir (max-age=60 — 60 saniye boyunca tekrar sorulmadan önbellekten kullanılabilir).
- **ETag**: Kaynağın belirli bir sürümünün "parmak izi"dir; istemci bir sonraki istekte If-None-Match başlığıyla bu ETag'i gönderirse, sunucu veri değişmediyse sadece 304 Not Modified döner (gövdeyi tekrar göndermeden bant genişliği tasarrufu sağlar).

SONUÇ

HTTP, tüm Web API'lerinin ortak dilidir: istek/yanıt yapısı, metodlar (filler), durum kodları ve üst bilgiler, bir API ile konuşurken kullanacağımız temel kelime dağarcığını oluşturur. Bir sonraki dosyada (22.3.1), bu temel taşları kullanarak İYİ bir API'nin nasıl TASARLANACAĞINI — yani REST mimarisi prensiplerini, kaynak modellemesini ve sayfalama stratejilerini — inceleyeceğiz.